

České vysoké učení technické v Praze
Fakulta jaderná a fyzikálně inženýrská

Katedra softwarového inženýrství
Program: Aplikace informatiky v přírodních vědách
Specializace: –



Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

DIPLOMOVÁ PRÁCE

Vypracoval: Bc. Martin Johec
Vedoucí práce: Ing. Josef Nový, Ph.D.
Rok: 2025

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Jochec** Jméno: **Martin** Osobní číslo: **502714**
Fakulta/ústav: **Fakulta jaderná a fyzikálně inženýrská**
Zadávací katedra/ústav: **Katedra softwarového inženýrství**
Studijní program: **Aplikace informatiky v přírodních vědách**

II. ÚDAJE K DIPLOMOVÉ PRÁCI

Název diplomové práce:

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Název diplomové práce anglicky:

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

Pokyny pro vypracování:

1. Nastudujte literaturu týkající se strojového učení v herních prostředích a adaptace agentů.
2. Implementujte předem navržené algoritmy a experimentálně je otestujte ve vhodném herním simulátoru.
3. Vylepšete navržené algoritmy s cílem zvýšit efektivitu adaptace agentů v dynamických prostředích.
4. Vyhodnoťte výsledky experimentů a porovnejte efektivitu jednotlivých verzí algoritmů adaptace agentů v implementovaném herním simulátoru.

Seznam doporučené literatury:

1. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). Cambridge: MIT Press. ISBN 978-0262039246.
2. Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Hoboken: Pearson. ISBN 978-0134610993.
3. Abbeel, P., & Schulman, J. (2016). Continuous control with deep reinforcement learning. arXiv preprint. Dostupné z: <https://arxiv.org/abs/1509.02971>.
4. Liébana, D. P., Lucas, S. M., et al. (2020). General Video Game Artificial Intelligence. Springer International Publishing. ISBN 978-3031021220.

Jméno a pracoviště vedoucí(ho) diplomové práce:

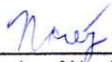
Ing. Josef Nový, Ph.D. Katedra softwarového inženýrství FJFI Děčín

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) diplomové práce:

Datum zadání diplomové práce: **29.10.2024**

Termín odevzdání diplomové práce: **09.05.2025**

Platnost zadání diplomové práce: **28.10.2026**


Ing. Josef Nový, Ph.D.
podpis vedoucí(ho) práce


podpis vedoucí(ho) ústavu/katedry


doc. Ing. Václav Čuba, Ph.D.
podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Diplomant bere na vědomí, že je povinen vypracovat diplomovou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v diplomové práci.

31.10.2024
Datum převzetí zadání


Podpis studenta

PROHLÁŠENÍ

Já, níže podepsaný(á)

Příjmení, jméno studenta: Johec Martin

Osobní číslo: 502714

Název programu: Aplikace informatiky v přírodních vědách

prohlašuji, že jsem bakalářskou/diplomovou práci s názvem

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

.....

vypracoval(a) samostatně a uvedl(a) veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací a Rámcovými pravidly používání umělé inteligence na ČVUT pro studijní a pedagogické účely v Bc a NM studiu.

☒ Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil/a nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil/a. Stvrzuji, že jsem si vědom/a, že za obsah závěrečné práce plně zodpovídám.

☐ Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce nepoužil/a žádný nástroj umělé inteligence. Jsem si vědom/a důsledků, kdy bude zjevné nepřiznané použití těchto nástrojů při tvorbě jakékoli části mé závěrečné práce.

v Praze dne 7.4.2025

.....
Johec
Podpis

Poděkování

Děkuji Ing. Josefu Novému, Ph.D., za vedení mé diplomové práce a za případnou pomoc při její tvorbě.

Bc. Martin Johec

Název práce:

Algoritmy strojového učení pro adaptaci autonomních agentů v 3D akční hře

Autor: Bc. Martin Jochec

Studijní program: Aplikace informatiky v přírodních vědách

Specializace: –

Druh práce: Diplomová práce

Vedoucí práce: Ing. Josef Nový, Ph.D.

Katedra softwarového inženýrství, Fakulta jaderná a fyzikálně inženýrská, České vysoké učení technické v Praze

Konzultant: –

Abstrakt:

Klíčová slova: hry, umělá inteligence, adaptace, autonomní agent, strojové učení

Title:

Machine Learning Algorithms for Adapting Autonomous Agents in a 3D Action Game

Author: Bc. Martin Jochec

Abstract:

Key words: games, artificial intelligence, adaptation, autonomous agent, machine learning

Obsah

Úvod	9
1 Samoučící se AI ve hrách	11
1.1 Současné aplikace samoučící se AI ve hrách	11
1.1.1 AlphaStar	11
1.1.2 OpenAI Five	14
1.1.3 Voyager	16
1.2 Algoritmy a techniky používané pro samoučící se agenty	18
1.2.1 Proximal Policy Optimization (PPO)	18
1.2.2 Self-Play	21
1.2.3 Population Based Training	23
2 Návrh, implementace a testování hry a autonomního agenta	25
2.1 Návrh a implementace prostředí hry	25
2.2 Návrh a implementace autonomního agenta	25
2.3 Otestování chování agenta	25
3 Experimentování s modifikacemi agenta	27
4 Vyhodnocení různých implementací agenta	29
Závěr	31
Literatura	32
Přílohy	35
A Název první přílohy	35

Úvod

Kapitola 1

Samoučící se AI ve hrách

1.1 Současné aplikace samoučící se AI ve hrách

1.1.1 AlphaStar

AlphaStar[11] je agent umělé inteligence (AI), který byl vyvinut společností DeepMind pro hraní StarCraft II[15], známé a dynamické online strategie v reálném čase od společnosti Blizzard Entertainment. Projekt byl poprvé představen veřejnosti v lednu 2019. Již v srpnu téhož roku AlphaStar dosáhl hodnosti „Grandmaster“, což představovalo významný milník nejen v oblasti AI zaměřené na videohry, ale do jisté míry i pro celý obor vývoje inteligentních systémů.

Hry navržené pro lidské hráče slouží jako cenná měřítka pokroku v oblasti umělé inteligence, jelikož nabízejí komplexní a dynamická prostředí s externí validitou. Zatímco dřívější milníky, jako Deep Blue[14] od IBM nebo AlphaGo od DeepMind[1], ukázaly schopnosti AI v tahových hrách s úplnou informací, StarCraft přináší nové výzvy díky hraní v reálném čase, částečné pozorovatelnosti, absenci jediné optimální strategie a rozsáhlému prostoru možných akcí. Přesto je mezi výzkumníky považován za řešitelný s využitím moderních technologií, a to díky svému diskrétnímu charakteru, známým pravidlům a omezenému počtu objektů, což z něj činí ideální testovací prostředí pro agenty, jako je AlphaStar.

DeepMind Technologies byla založena ve Spojeném království v roce 2010. Již v roce 2011 spoluzakladatel Demis Hassabis označil StarCraft za „další krok vpřed“ po hrách jako Go. Poté, co DeepMind předvedl schopnosti svých agentů na základě posilovaného učení (Reinforcement Learning), kteří překonali lidské hráče v klasických hrách na Atari 2600, byla společnost v roce 2014 odkoupena firmou Google, což otevřelo cestu k dalším ambiciózním projektům. Jedním z nich byl právě vývoj AI pro StarCraft II, o který zájem postupně rostl.

Počítačový vědec Zachary Mason v roce 2015 předpověděl, že výzkum DeepMind by mohl během pěti až deseti let vést k průlomu právě v této hře. Myšlenka získala větší váhu v roce 2016, kdy AlphaGo porazil světového šampiona v Go, Lee Sedola. Po tomto úspěchu Hassabis veřejně projevil zájem vytvořit autonomního agenta pro StarCraft, přičemž zdůraznil jeho strategickou hloubku, částečnou pozorovatelnost, skryté informace a vlastnosti, které ho činí ještě náročnějším než Go.

Formální spolupráce mezi DeepMind a Blizzardem byla oznámena na BlizzConu v listopadu 2016, spolu s plánem na vydání otevřeného vývojového prostředí pro boty na začátku roku 2017. Do poloviny roku 2017 už DeepMind začal zpracovávat herní data ze StarCraft II. V srpnu téhož roku pak obě společnosti uvolnily nástroje pro vývoj botů a soubor 65000 záznamů z profesionálních zápasů. Odborníci tehdy odhadovali, že potrvá ještě několik let, než AI porazí špičkového hráče. Tyto předpovědi se však ukázaly jako příliš opatrné, protože v prosinci 2018 porazil agent od DeepMind profesionálního hráče Grzegorze „MaNu“ Komincze poměrem 5:0. V lednu 2019 byl tento agent oficiálně představen veřejnosti pod jménem „AlphaStar“. Navzdory tomuto úspěchu se však objevila kritika.

AlphaStar měl výhodu v podobě přímé komunikace s hrou přes API, což mu umožňovalo provádět akce s chirurgickou přesností. Dále měl globální přehled o mapě místo omezení herní kamerou a mohl v rámci daného časového okna rozložit akce nerovnoměrně, čímž byl schopen vyvinout extrémní tlak v klíčových momentech. DeepMind na tyto připomínky reagoval přeškolením AlphaStar pod realističtějšími omezeními, která více odpovídala lidským možnostem. V následném zápase proti MaNově už tato verze prohrála.

Od července 2019 začal upravený AlphaStar anonymně soutěžit na veřejném evropském žebříčku v režimu 1v1, kde se utkával s hráči, kteří s tímto experimentem souhlasili. Na konci srpna 2019, jak již bylo zmíněno, dosáhl AlphaStar hodnosti „Grandmaster“, čímž se zařadil mezi nejlepší 0,2% lidských hráčů a potvrdil tak významný pokrok v oblasti autonomních agentů pro komplexní hry v reálném čase.

Zatímco zpětná propagace (Backpropagation, BP) je stále hlavní metodou trénování neuronových sítí, AlphaStar využívá pokročilejší přístup zvaný Population-Based Training (PBT)[4]. Tento přístup kombinuje výhody evolučních algoritmů (EAs), které slouží k průzkumu globálního prostoru řešení, s lokální optimalizací pomocí BP. AlphaStar konkrétně používá Lamarckovskou evoluci, kde úspěšné sítě předávají své parametry a upravené hyperparametry méně výkonným sítím.

Pro zvládnutí vysoké výpočetní náročnosti je PBT navržen jako asynchronní a distribuovaný systém, což umožňuje paralelní trénink agentů bez nutnosti synchronizace. Na rozdíl od klasických generačních evolučních algoritmů využívá PBT tzv. steady-state přístup, kdy jsou agenti průběžně aktualizováni bez zastavení procesu. Tento mechanismus zvyšuje efektivitu, udržuje diverzitu a umožňuje průběžné přizpůsobování hyperparametrů—což je klíčové v prostředích s proměnlivým tréninkovým chováním, jako je hluboké posilované učení.

AlphaStar dále využívá kompetitivní koevoluci, při níž se agenti neučí pouze proti svým předchozím verzím, ale proti celé populaci různorodých protivníků. Tento přístup poskytuje přirozenou tréninkovou křivku a zároveň zvyšuje robustnost strategií, jelikož agent musí zvládnout široké spektrum herních stylů. Výběr soupeřů se provádí na základě podobnosti výkonnosti (např. Elo skóre), což umožňuje plynulý rozvoj bez stagnace.

Proces učení a rozhodování AlphaStaru lze ilustrovat na konkrétním příkladu herní situace, kde agent zpracovává aktuální stav mapy, aktivuje neuronové sítě a vyhodnocuje dostupné možnosti. Vizualizace tohoto procesu je znázorněna na obrázku 1.1.



Obrázek 1.1: Vizualizace rozhodovacího procesu AlphaStaru během zápasu proti hráči. Systém zpracovává syrová pozorování, aktivuje neuronové sítě, vyhodnocuje potenciální akce a předpovídá výsledky zápasu.

Protože StarCraft II nemá jednu univerzální vítěznou strategii, AlphaStar neusiluje o vytvoření jediného dokonalého agenta. Místo toho udržuje populaci různorodých strategií, vybraných na základě Nashovy distribuce—tedy takových, které jsou vůči sobě navzájem nejméně zranitelné.

Tento přístup odpovídá principům kvalitní diversity (Quality Diversity – QD). Místo hledání jediné nejlepší varianty QD algoritmy podporují vznik více kvalitních, ale odlišných řešení. V AlphaStaru jsou strategie určovány pomocí herně specifických charakteristik—například preferencí pro určité jednotky nebo schopností porazit konkrétní protivníky. Tyto charakteristiky se navíc během tréninku dynamicky upravují, čímž vzniká rozmanitější a odolnější sada řešení.

Díky tomuto přístupu AlphaStar operuje s portfoliem strategií, které jsou přizpůsobivé, méně náchylné k porážce a lépe si poradí s různorodostí lidských protivníků.

1.1.2 OpenAI Five

OpenAI Five[6] je agent umělé inteligence určený pro hraní týmové videohry Dota 2, ve které proti sobě nastupují dvě pětičlenná družstva. Poprvé se veřejnosti představil v roce 2017 při živé demonstraci, během níž porazil profesionálního hráče Dendiho v duelu jeden na jednoho. V následujícím roce systém pokročil natolik, že zvládl ovládat celý tým pěti agentů a porážet i profesionální lidské týmy.

Společnost OpenAI si zvolila Dota 2[3] kvůli její složitosti, nepředvídatelnosti a plynulému průběhu, který lépe odráží podmínky skutečného světa. Díky tomu byla hra vhodná pro vývoj univerzálnějších systémů pro řešení komplexních problémů. Algoritmy a kód vytvořené pro OpenAI Five byly později využity také v dalších projektech, například pro ovládání robotické ruky pomocí neuronové sítě. OpenAI Five bývá často zmiňován vedle jiných přelomových AI systémů, které porazily lidi v konkrétních úlohách—například AlphaStar ve StarCraft II, AlphaGo ve hře Go nebo Deep Blue v šachu.

Vývoj algoritmů, na nichž je OpenAI Five postaven, začal v listopadu 2016. OpenAI si zvolila Dota 2 jako testovací prostředí mimo jiné kvůli popularitě hry na platformách jako Twitch, nativní podpoře Linuxu a dostupnosti aplikačního rozhraní (API).

První významná veřejná demonstrace proběhla v srpnu 2017 na turnaji The International, což je každoroční vrcholná událost ve světě Dota 2. Během této akce porazil raný model bota profesionálního hráče Dendiho v duelu jeden na jednoho. Podle technického ředitele OpenAI Grega Brockmana se systém učil metodou posilovaného učení (reinforcement learning), přičemž si své schopnosti zlepšoval hraním sám proti sobě po dobu dvou týdnů. V tomto režimu agent získával odměny za klíčové akce ve hře, jako je zabíjení nepřátel nebo ničení struktur.

V červnu 2018 už byl systém rozšířen do plnohodnotného pětičlenného týmu, který dokázal porážet amatérské a poloprofesionální lidské soupeře. Na The International 2018 se OpenAI Five zúčastnil dvou exhibičních zápasů proti týmům na profesionální úrovni—proti brazilskému paiN Gaming a proti výběru bývalých elitních čínských hráčů. Přestože boti obě utkání prohráli, zkušenost byla pro vývojový tým cenná, protože umožnila další ladění strategií a rozhodovacích mechanismů.

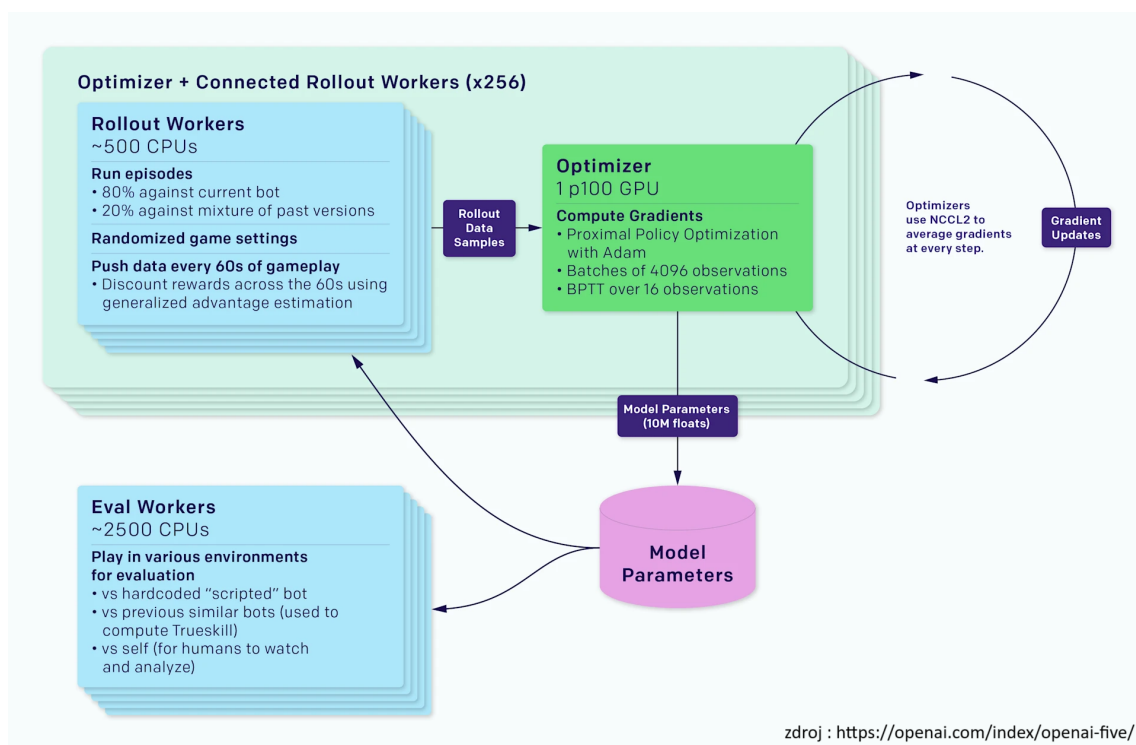
Závěrečná veřejná prezentace projektu proběhla v dubnu 2019, kdy OpenAI Five porazil v sérii na dvě vítězství tým OG, tehdejší mistry světa z turnaje The International 2018. Ve stejném měsíci proběhla i čtyřdenní online událost otevřená veřejnosti, během níž se systém zúčastnil více než 42 000 zápasů—z nichž vyhrál přibližně 99,4%.

OpenAI Five je založen na rámci hlubokého posilovaného učení, jehož jádrem je metoda Proximal Policy Optimization (PPO)[7][9]—gradientní algoritmus politiky, který vyvažuje stabilitu učení, jednoduchost implementace a efektivitu využití dat. PPO představuje vylepšení oproti dřívějším metodám, jako jsou běžné policy gradients nebo Trust Region Policy Optimization (TRPO), a to díky zavedení tzv. „clipped surrogate“ objektivní funkce, která zabráňuje příliš razantním aktualizacím politiky a zajišťuje konzistentní pokrok při tréninku.

Každý agent je reprezentován neuronovou sítí s jednou vrstvou typu LSTM o 4096 jednotkách, která zpracovává informace o aktuálním stavu hry pomocí API poskytnutého vývojáři Dota 2. Tento stav je zakódován jako vektor přibližně 20 000 číselných hodnot, které popisují prostředí z pohledu daného agenta.

Rozhodování probíhá prostřednictvím několika nezávislých akčních hlav, z nichž každá odpovídá za jiný aspekt chování—například volbu konkrétní akce, její umístění na prostorové mřížce nebo zpoždění provedení o určitý počet ticků. Tyto hlavy vytvářejí osm výstupních hodnot pro každé rozhodnutí, což umožňuje širokou škálu koordinovaných akcí bez nutnosti využívat lidská trénovací data.

Trénink probíhal na interní infrastruktuře Rapid, která umožňuje rozsáhlý paralelní výpočet a koordinaci tisíců strojů. Díky tomu mohl OpenAI Five provádět trénink skrze masivní self-play. Do roku 2018 systém odehrál ekvivalent více než 180 let herního času, a to na 256 GPU a 128 000 CPU jádrech. Schéma tohoto výpočetního procesu je znázorněno na obrázku 1.2.



Obrázek 1.2: Architektura tréninkového systému OpenAI Five. Rollout workeri generují data ze zápasů, která jsou následně zpracována optimalizačním modulem využívajícím metodu PPO. Výsledné aktualizace jsou sdíleny napříč instancemi a evaluovány ve specifických herních prostředích.

Díky své robustnosti, efektivitě a jednoduché integraci do distribuovaných systémů byl PPO mimořádně vhodný pro tuto aplikaci. Umožnil rozsáhlý paralelní trénink bez přeučení, přičemž jeho kompatibilita se sdílenými politikami, hodnotovými funkcemi a rekurentními vrstvami typu LSTM přispěla k jeho úspěchu v komplexním prostředí, jakým je Dota 2.

1.1.3 Voyager

Voyager[12] je agent umělé inteligence, který propojuje výhody velkých jazykových modelů (LLM) se schopností samostatného učení ve složitých, otevřených prostředích. Na rozdíl od předchozích přístupů, které spoléhají na ručně vytvořené cíle, odměnové funkce nebo specificky upravená prostředí, Voyager operuje v původní verzi hry Minecraft, a to bez jakékoli lidské intervence nebo ručního ladění. Jedná se tak o příklad tzv. ztělesněného agenta (embodied agent), který nejen interpretuje herní svět, ale v něm i fyzicky jedná, sbírá zkušenosti a postupně se zlepšuje.

Výběr hry Minecraft nebyl náhodný. Jde o jedno z mála prostředí, které kombinuje otevřenost, dlouhodobé cíle, nutnost plánování, nelineární obtížnost a procedurálně generované světy. Neexistuje zde žádný „konec hry“, a úspěch se nedá snadno kvantifikovat. To činí z Minecraftu ideální testovací prostředí pro dlouhodobé učení bez explicitní odměny. Agent musí samostatně odhalit základní principy světa – od těžby a výroby nástrojů, přes přežití v noci, až po složité postupy jako farmaření nebo výroba lektvarů.

Aby toho dosáhl, Voyager využívá kombinaci tří klíčových mechanismů, které společně umožňují otevřené, samostatné a škálovatelné učení. Prvním z nich je automatická tvorba kurikula. Agent si sám generuje posloupnost úkolů pomocí velkého jazykového modelu GPT-4[2], přičemž tyto úkoly jsou vždy odvozeny ze současného stavu světa, inventáře, vybavení, aktuální lokace a historie předchozích úspěchů i selhání. Místo toho, aby agent postupoval podle předem definovaného scénáře, sám si formuluje dosažitelné, smysluplné cíle – například „vytvoř kamenný krumpáč“, „postav pec“, nebo „zabij zombíka“.

Tyto úkoly nejsou náhodné. GPT-4 při jejich generování zohledňuje různé faktory, jako je biome, čas dne, přítomnost entit v okolí nebo například to, jaké úkoly již byly splněny. Díky tomu agent postupuje přirozeným tempem, které reflektuje jeho aktuální schopnosti a možnosti. Tento mechanismus tak vytváří dynamické, adaptivní kurikulum, které maximalizuje diverzitu dovedností bez nutnosti ručně nastavovat, co má být cílem.

Každý úkol, který se agentovi podaří splnit, je následně převeden do podoby samostatného programu napsaného v JavaScriptu. Tyto programy představují „dovednosti“ (skills), které Voyager ukládá do tzv. knihovny dovedností. Tato knihovna není statická – každá nově naučená dovednost je indexována vektorově pomocí embeddingu popisu, který vytvoří jazykový model GPT-3.5[2]. Tím vzniká databáze, ve které může agent v budoucnu vyhledávat relevantní programy, jež mu pomohou vyřešit nové úkoly.

Dovednosti jsou přitom konstruovány hierarchicky. Složitější akce (např. „vytvoř železný krumpáč“) využívají jednodušší dovednosti jako „vypal železnou rudu“, která sama závisí na dovednosti „těž železnou rudu“ a „vyrob pec“. Tento kompoziční přístup umožňuje agentovi stavět na tom, co se již naučil, a minimalizuje nutnost začínat stále od nuly. Zároveň se tím eliminuje problém tzv. katastrofického zapomínání, který je běžný u běžných metod posilovaného učení.

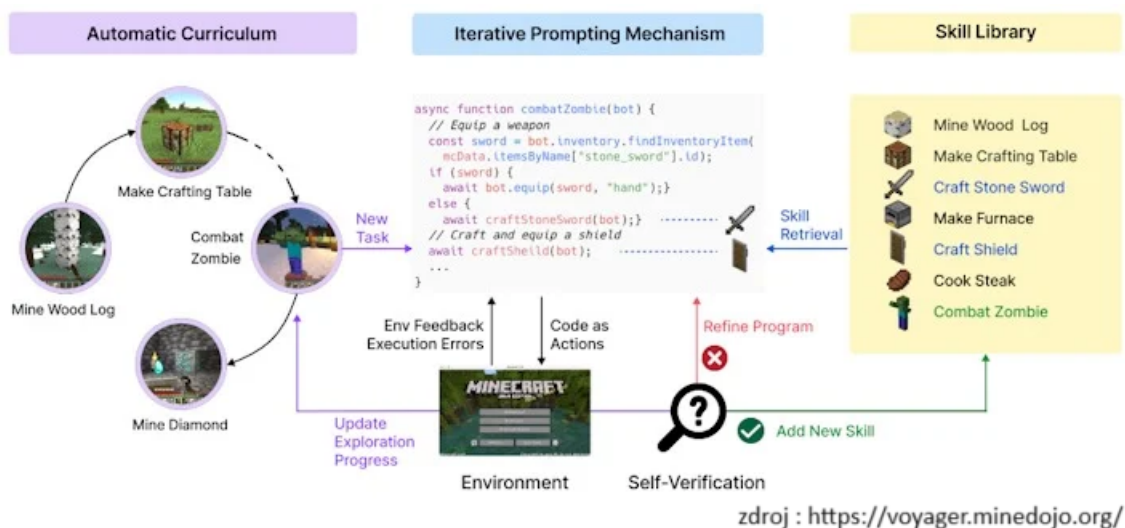
V rámci samotného učení Voyager nevyužívá klasický přístup typu „pozorování →

akce“, ale spoléhá na iterativní promptování. Při řešení úkolu agent nejprve navrhne program, který by měl daný úkol vyřešit. Tento program se následně provede v prostředí a výstup – včetně chyb a výjimek – je znovu předložen GPT-4. Na základě této zpětné vazby model prompt upraví, program opraví a zkusí jej znovu. Tento proces se opakuje, dokud tzv. sebe-verifikátor, opět založený na GPT-4, nevyhodnotí, že byl úkol úspěšně dokončen.

Iterativní přístup se ukázal jako klíčový zejména v případech, kdy agent narazí na nekonzistence mezi plánem a realitou. Například pokud program selže s chybou typu „I cannot make a stick because I need: 2 more planks“, systém je schopen automaticky provést korekci a aktualizovat plán. Pokud se problém opakuje (čtyři po sobě jdoucí neúspěšné pokusy), agent daný úkol opustí a vygeneruje si nový.

Ve srovnání s předchozími přístupy, jako jsou ReAct, Reflexion nebo AutoGPT, dosáhl Voyager výrazně lepších výsledků. V prostředí MineDojo získal $3,3\times$ více unikátních předmětů, prozkoumal svět do větší vzdálenosti ($2,3\times$ delší trasy) a dosáhl technologického pokroku až $15,3\times$ rychleji. Navíc jako jediný agent zvládl tzv. zero-shot úlohy, kde byl přenesen do nového světa s novým úkolem, který dosud neřešil – např. vytvoření dalekohledu nebo nalezení diamantů.

Díky kombinaci autonomního generování úkolů, kompoziční knihovny dovedností a schopnosti iterativní korekce chyb se Voyager přibližuje představě agenta, který je schopen dlouhodobého, otevřeného učení bez potřeby lidského zásahu. Schéma na obrázku 1.3 přehledně znázorňuje propojení těchto tří klíčových komponent. V prostředí, které není explicitně navrženo pro AI trénink, vykazuje chování, které je zároveň zvědavé, adaptivní i efektivní – a představuje tak významný krok k obecné umělé inteligenci v otevřeném světě.



Obrázek 1.3: Vizualizace propojení hlavních komponent systému Voyager.

1.2 Algoritmy a techniky používané pro samoučící se agenty

1.2.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO)[7][9] je jedním z nejrozšířenějších algoritmů pro trénování agentů pomocí hlubokého posilovaného učení. Vyvinut byl výzkumníky ze společnosti OpenAI jako reakce na klíčové nedostatky tehdejších metod policy gradient algoritmů, které často provádějí příliš velké aktualizace politiky bez jakékoli kontroly, a složitější metody jako Trust Region Policy Optimization (TRPO)[8], které jsou výpočetně náročné a obtížně implementovatelné. PPO nabízí kompromis: spojuje jednoduchost implementace s vysokou stabilitou učení a efektivním využitím dat.

Algoritmus PPO patří do rodiny metod založených na optimalizaci politiky. Na rozdíl od metod hodnotové funkce, jako je Q-learning[13], přímo modeluje politiku agenta (pravděpodobnostní rozdělení nad akcemi) a učí se ji optimalizovat tak, aby maximalizoval dlouhodobou očekávanou odměnu. PPO navazuje na TRPO, který zavádí omezení velikosti aktualizace politiky, aby se zabránilo destruktivním skokům ve strategii agenta, ale činí tak výrazně jednodušším a výpočetně efektivnějším způsobem.

Základní myšlenka PPO spočívá v tom, že při každé aktualizaci politiky se zamezuje příliš velkým změnám pravděpodobnostních rozdělení nad akcemi. Konkrétně se zavádí tzv. *clipped surrogate objective*, která upravuje klasický výraz pro policy gradient, aby penalizovala změny mimo stanovený rozsah. Tento přístup umožňuje provádět více kroků stochastického gradientního vzestupu na stejných datech bez rizika destabilizace učení.

Matematicky lze optimalizovaný výraz popsat jako:

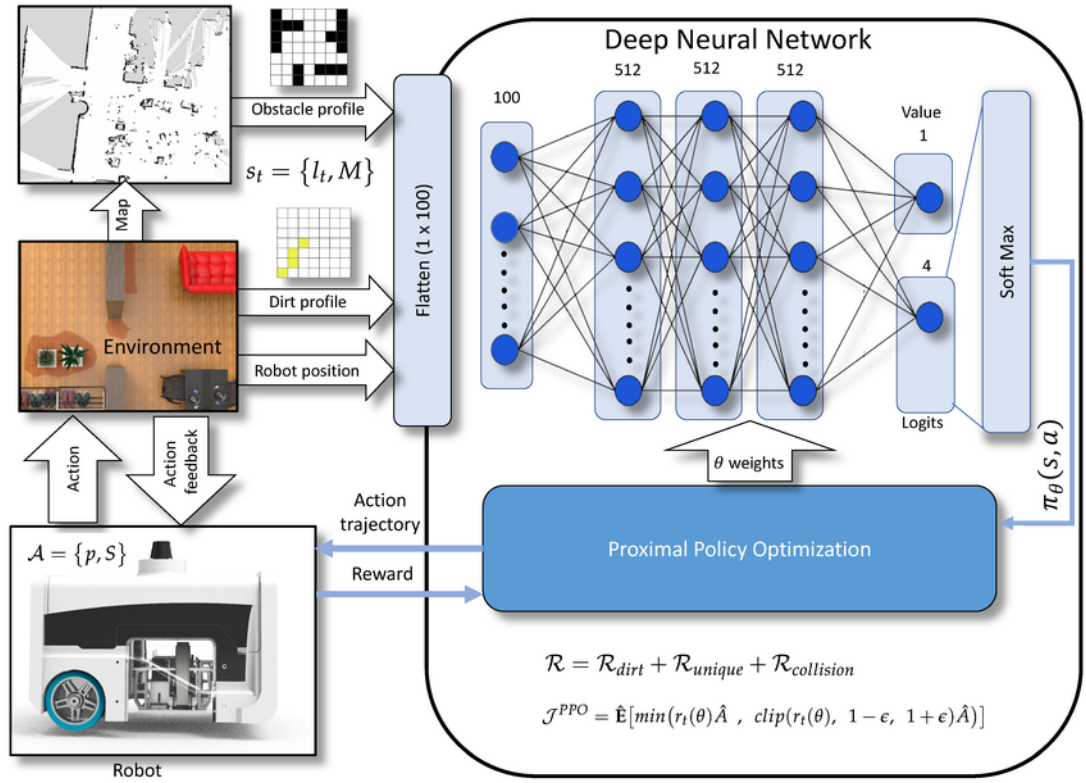
$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right],$$

kde

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

je poměr pravděpodobností podle nové a staré politiky a $\hat{A}_t$ je odhad tzv. advantage funkce, která kvantifikuje, jak výhodná byla daná akce vzhledem k očekávanému vývoji. Hodnota ϵ je hyperparametr, typicky nastavený na 0,1 až 0,3, který určuje toleranci změn politiky. Symbol $\hat{\mathbb{E}}_t$ představuje empirické očekávání vypočtené na základě vzorků nasbíraných během interakcí agenta s prostředím.

Smyslem výrazu *clip* je omezit motivaci agenta k tomu, aby příliš radikálně měnil svou strategii, pokud taková změna vede k výraznému zvýšení pravděpodobnosti akcí. Pokud je změna politiky příliš výrazná (např. $r_t > 1 + \epsilon$), je výpočet výsledku „oříznut“ tak, aby nevedl k nadhodnocení zisku. Naopak, pokud je změna v rámci povoleného pásma, je výpočet ponechán. Výsledkem je stabilnější učení, které se vyhýbá oscilacím a přetížení aktualizací. Celý proces je vizuálně shrnut na obrázku 1.4, který zachycuje architekturu PPO včetně interakce s prostředím, výpočtu hodnoty akce a aktualizace politiky.



zdroj : https://www.researchgate.net/figure/The-reinforcement-learning-architecture-based-on-Proximal-Policy-Optimization_fig3_357019705

Obrázek 1.4: Vizualizace architektury algoritmu PPO. Agent interaguje s prostředím, zatímco dvě neuronové sítě — actor a critic — slouží k výběru akcí a odhadu hodnoty stavu. Politika je aktualizována pomocí oříznutého objektivu a výhod jsou počítány pomocí Generalized Advantage Estimation (GAE).

Kromě této hlavní ztrátové funkce lze PPO dále rozšířit o dvě další složky — penalizaci chyby odhadu hodnoty a entropickou regulaci. Celková ztrátová funkce pak má tvar:

$$L_t^{\text{PPO}}(\theta) = \hat{\mathbb{E}}_t [L_t^{\text{CLIP}}(\theta) - c_1 L_t^{\text{VF}}(\theta) + c_2 \mathcal{S}(\pi_\theta, s_t)],$$

kde L_t^{VF} je kvadratická chyba hodnotové funkce, $\mathcal{S}(\pi_\theta, s_t)$ je entropie distribuční politiky ve stavu s_t a c_1, c_2 jsou váhové koeficienty. Přidání entropické složky pomáhá zajistit průzkum prostředí (exploration), zejména v počátečních fázích tréninku, a zabráňuje tomu, aby se agent „zasekl“ v lokálním maximu.

Pro výpočet výše zmíněné advantage funkce $\hat{A}_t$ se často používá metoda Generalized Advantage Estimation (GAE)[10]. Ta snižuje rozptyl odhadů tím, že kombinuje okamžité odměny a odhady hodnoty pomocí exponenciálního vážení parametrizovaného faktorem λ :

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \dots,$$

kde

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

a γ je diskontní faktor. GAE tak umožňuje lepší kompromis mezi zkreslením a rozptylem, což se v praxi promítá do rychlejšího a stabilnějšího učení.

Implementace PPO obvykle probíhá ve stylu tzv. *actor-critic*. Paralelní agenti (herní instance) generují trajektorie, typicky v rozsahu stovek časových kroků, a výsledné vzorky se následně použijí pro více iterací minibatch SGD nad ztrátovou funkcí. Při tréninku se často používá optimalizátor Adam a architektura s oddělenou politikou a hodnotovou funkcí, i když v některých případech lze využít i sdílené parametry.

Z praktického hlediska byl PPO úspěšně aplikován na širokou škálu prostředí. V oblasti spojitě kontroly (např. úkoly v prostředí MuJoCo[5]) překonal algoritmus PPO jiné tehdy běžně používané přístupy jako TRPO, A2C nebo CEM, a to jak z hlediska dosažené odměny, tak z hlediska rychlosti konvergence. Rovněž ve hrách z prostředí Atari, kde byl porovnáván s A2C a ACER, ukázal PPO vysokou robustnost napříč různými herními scénáři. Navíc oproti metodám s pevným KL penalizačním koeficientem dosahoval stabilnějších výsledků díky použití tzv. *adaptive KL penalty* nebo oříznutí podle výše uvedeného pravidla.

PPO se stal preferovaným algoritmem v mnoha aplikacích nejen díky výkonu, ale i díky snadné integraci do různých tréninkových rámců. Kromě empirických výsledků je výhodou PPO i jeho jednoduchost. Z hlediska implementace vyžaduje pouze mírnou úpravu klasického policy gradient frameworku a nevyžaduje řešení složitých optimalizačních úloh jako TRPO (např. pomocí konjugovaného gradientu). Díky tomu se stal PPO standardní volbou v mnoha open-source knihovnách posilovaného učení, včetně OpenAI Baselines, Stable Baselines nebo RLlib.

Celkově lze PPO vnímat jako rozumný kompromis mezi výpočetní náročností, robustností a výkonem. Jeho úspěch spočívá v jednoduché, ale účinné úpravě klasických policy gradient metod, která zajišťuje, že každá aktualizace politiky zůstane „blízko“ té původní, čímž se zamezí katastrofálním změnám a zároveň se umožní efektivní učení.

1.2.2 Self-Play

Self-play[16] je technika v oblasti posilovaného učení, která umožňuje agentům zlepšovat své strategie prostřednictvím interakce se sebou samými — tedy s kopiemi, staršími verzemi nebo současnými protivníky v rámci jednoho tréninkového rámce. Tato metoda se ukázala jako zásadní pro dosažení vyšších výkonů ve hrách, jako jsou Go, šachy, StarCraft II nebo Dota 2. Na rozdíl od klasického posilovaného učení, kde agent interaguje s předem daným prostředím, zde prostředí zahrnuje další agentní entity, jejichž chování se v čase mění — a tím se vytváří dynamický a nelineární tréninkový proces.

Základní přístup self-play spočívá v tom, že agent trénuje proti svým vlastním kopiím, ať už současným, nebo historickým. Nejjednodušší forma, tzv. „vanilla self-play“, spočívá v tom, že se agent neustále střetává s poslední verzí sebe sama. Tato metoda je velmi efektivní v tranzitivních hrách, kde lze jednotlivé strategie jednoznačně seřadit dle síly, avšak v nestranově tranzitivních prostředích (například kámen-nůžky-papír) vede často ke cyklickému chování a může konvergovat k suboptimálním strategiím.

Aby se tyto nedostatky zmírnily, vznikly složitější varianty. „Fiktivní self-play“ rozšiřuje trénink tak, že agent hraje proti průměru všech svých předchozích verzí. Tím se posiluje robustnost učení a eliminuje riziko zacyklení. „Prioritizovaná fiktivní self-play“ (např. použitá v AlphaStar) dále zavádí vážený výběr protivníků, čímž zvýhodňuje ty, proti nimž si agent vede hůře — tedy záměrně trénuje proti „slabým místům“ své strategie.

Další variantou je „ δ -uniform self-play“, která používá hyperparametr δ k určení, jaké části historické populace budou použity jako soupeři. Nižší δ znamená širší spektrum historických politik, vyšší hodnoty naopak preferují nedávné soupeře.

Moderní rámce self-play algoritmů se často integrují s tzv. „population-based“ přístupy. Místo jednoho agenta existuje populace politik Π , která se v čase rozšiřuje o nové verze. Každá nová politika je trénována vůči vzorku soupeřů, vybíraných z Π na základě určité metastrategie Σ , často reprezentované interakční maticí. Tento proces vytváří bohatý prostor pro evoluci strategií a vzájemnou adaptaci.

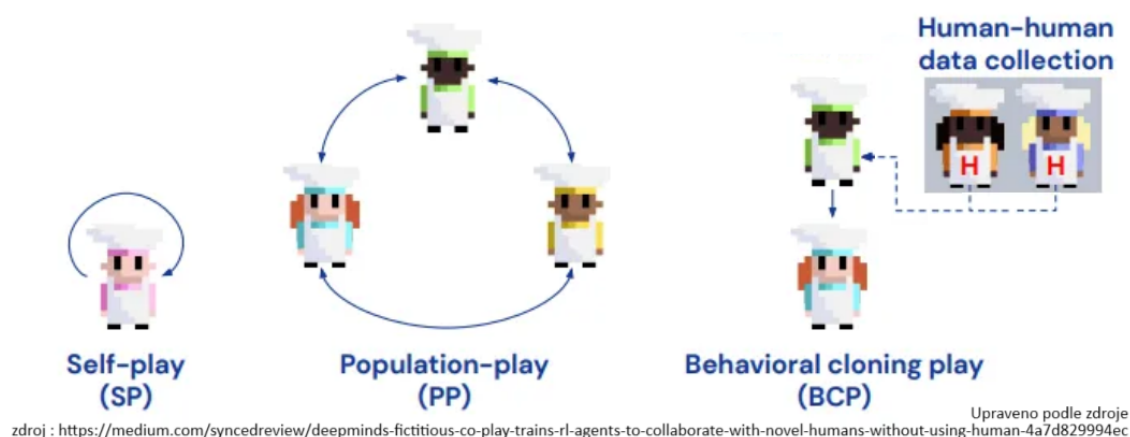
Základní tréninková smyčka obvykle probíhá následovně:

1. Nové politiky jsou inicializovány, často na základě posledních iterací předchozích politik.
2. Každá politika je trénována proti soupeřům vybraným podle Σ .
3. Nově natrénované politiky se přidají do populace a proces se opakuje.

Z hlediska výpočetní efektivity se rozlišuje mezi tzv. „lazy initialization“, kdy se politiky inicializují až v okamžiku potřeby, a „immediate initialization“, kdy je celá populace vytvořena hned na začátku. Dále rozlišujeme „standardní trénink“, kde se v každé iteraci trénuje pouze jeden agent, a „průběžný trénink“ (např. FTW agent), kde jsou všechny politiky trénovány souběžně.

Zvláštní pozornost si zaslouží tzv. PSRO (Policy-Space Response Oracles), které rozšiřují klasické self-play o výpočet rovnovážných strategií v rámci tzv. meta-herního prostoru. Místo toho, aby se agent učil proti jednotlivým politikám, PSRO používá například Nashovu rovnováhu jako metastrategii, vůči níž se hledají nejlepší odpovědi.

Kromě klasických her našla self-play široké uplatnění i v komplexních videohrách a reálných scénářích, například v robotice, autonomním řízení nebo syntéze proteinů. Také se používá, zatím v menší míře, v kontextu velkých jazykových modelů, například pro sebezdokonalování (self-improvement) nebo samostatnou konstrukci výukových dat. Základní rozdíl mezi přístupy self-play, population-play a behavioral cloning je naznačen na obrázku 1.5.



Obrázek 1.5: Srovnání tří přístupů k tréninku agentů: „self-play“ (SP), kde agent hraje sám proti sobě; „population-play“ (PP), kde interaguje s různými agenty v populaci; a „behavioral cloning play“ (BCP), kde se učení zakládá na lidských demonstračních datech.

1.2.3 Population Based Training

Population-Based Training (PBT)[4] je technika, která propojuje samotný trénink neuronových sítí s automatickým laděním hyperparametrů. Místo toho, aby se model trénoval s pevně danou konfigurací a po dokončení se hledala optimální nastavení v dalších bězích, PBT tento proces spojuje — optimalizuje váhy i hyperparametry zároveň a průběžně. To je zvláště užitečné v oblastech, kde jsou optimální hyperparametry v čase proměnlivé, například u posilovaného učení nebo generativních modelů.

Na začátku PBT inicializuje populaci modelů, kde každý má své vlastní váhy i hyperparametry. Všichni členové této populace jsou trénováni paralelně a nezávisle — často metodami jako gradient descent. Po určitém počtu kroků, nebo jakmile model dosáhne dostatečného výkonu, probíhá tzv. „exploit-and-explore“ fáze. Pokud je některý z modelů podprůměrný, může být jeho obsah nahrazen jiným, výkonnějším modelem z populace. Zároveň se na něj aplikuje lehce pozměněná verze jeho hyperparametrů — čímž vzniká nový „směr“ učení.

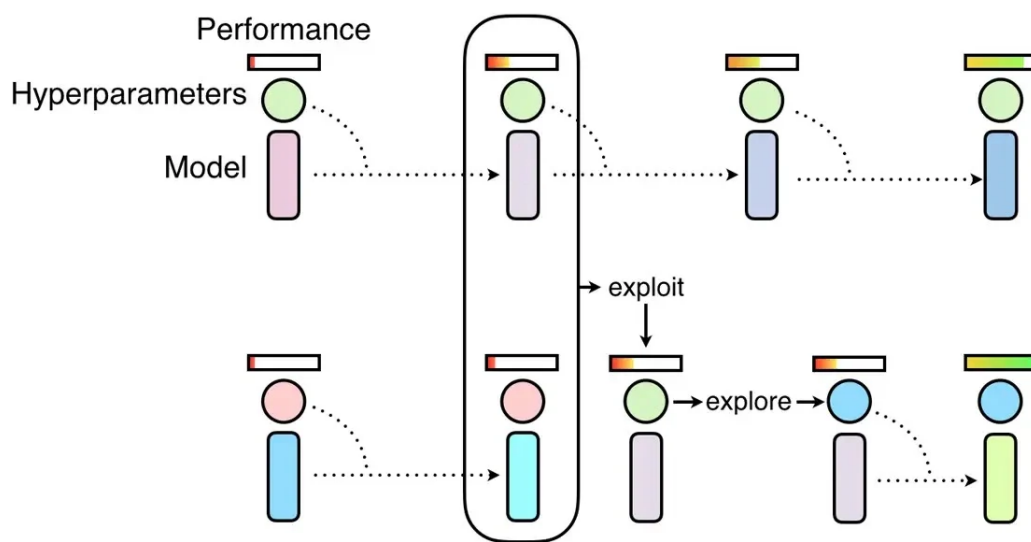
Tento přístup vytváří smyčku, ve které probíhá klasický trénink modelů, ale průběžně se mezi sebou také sdílejí informace a testují se nové kombinace. Tím dochází nejen k rychlejšímu konvergenci, ale také ke zlepšení finálního výkonu, protože hyperparametry nejsou fixní, ale průběžně se přizpůsobují aktuálnímu stavu tréninku.

Z praktického hlediska je PBT velmi dobře škálovatelný. Nevyžaduje centrální koordinaci — modely komunikují jen skrze jednoduché sdílení výkonových metrik a checkpointů. Díky tomu je možné PBT provozovat i na heterogenních nebo nestabilních výpočetních platformách, například v cloudovém prostředí s preemptibilními instancemi.

Základní smyčka algoritmu vypadá následovně:

1. Každý model ve své instanci trénuje standardními optimalizačními metodami (například SGD nebo Adam) pomocí aktuální konfigurace hyperparametrů.
2. Po určité době se jeho výkon vyhodnotí (například pomocí validace nebo herní odměny).
3. Pokud je model označen jako „ready“, provede se fáze „exploit“ — model si může zkopírovat váhy a hyperparametry od lepšího modelu v populaci.
4. Poté následuje fáze „explore“, kde jsou hyperparametry lehce pozměněny (například násobkem 1.2 nebo 0.8), případně náhodně přegenerovány.
5. Trénink pokračuje s těmito novými parametry.

Tento proces je znázorněn na obrázku 1.6, kde je vidět, jak modely mezi sebou sdílejí váhy a mění hyperparametry na základě výkonnosti.



zdroj : <https://deepmind.google/discover/blog/population-based-training-of-neural-networks/>

Obrázek 1.6: Population-Based Training neuronových sítí začíná jako náhodné prohledávání, ale v průběhu tréninku umožňuje jednotlivým modelům využít dílčí výsledky ostatních (fáze „exploit“) a zároveň zkoušet nové kombinace hyperparametrů (fáze „explore“).

Důležité je, že PBT není citlivý na přesné nastavení toho, kdy se fáze „exploit-and-explore“ provádí — obvykle stačí, aby mezi dvěma takovými událostmi uplynulo dost kroků na to, aby gradientové učení mělo čas ukázat efekt.

Technika PBT byla úspěšně aplikována v několika odlišných oblastech. V oblasti posilovaného učení pomohla například zrychlit trénink agentů v prostředí DeepMind Lab, StarCraft II nebo Atari. V těchto případech se automaticky ladily parametry jako learning rate, entropická penalizace nebo délka rozvinutí sítě (unroll length). V oblasti strojového překladu pomocí Transformerů PBT automaticky našlo hyperparametrové rozvrhy, které překonaly ručně laděné baseline modely. A u GANů PBT dokázalo zlepšit stabilitu tréninku i finální kvalitu generovaných obrázků.

Klíčovou výhodou PBT oproti klasickým metodám ladění (jako je grid search nebo Bayesian optimization) je, že probíhá „online“ — tedy současně s tréninkem. Díky tomu je výrazně efektivnější, protože modely nejsou trénovány od začátku znovu po každé změně hyperparametrů. PBT tak nabízí kombinaci výhod klasického evolučního přístupu a gradientového učení.

Celkově lze PBT vnímat jako techniku, která vnáší do optimalizace adaptivitu a sdílení znalostí v reálném čase. Je to metoda, která se výborně doplňuje s dalšími technikami, jako je self-play.

Kapitola 2

Návrh, implementace a testování hry a autonomního agenta

2.1 Návrh a implementace prostředí hry

2.2 Návrh a implementace autonomního agenta

2.3 Otestování chování agenta

Kapitola 3

Experimentování s modifikacemi agentů

Kapitola 4

Vyhodnocení různých implementací agentů

Závěr

Literatura

- [1] GOOGLE. *AlphaGo*. Online. GOOGLE. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/technologies/alphago/>. [cit. 2025-05-10].
- [2] CONTRIBUTORS, Wikipedia. *ChatGPT*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-, https://en.wikipedia.org/wiki/Main_Page. Dostupné z: <https://en.wikipedia.org/w/index.php?title=ChatGPT&oldid=1290307244>. [cit. 2025-05-10].
- [3] CONTRIBUTORS, Wikipedia. *Dota 2*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: https://en.wikipedia.org/w/index.php?title=Dota_2&oldid=1288870722. [cit. 2025-05-10].
- [4] JADERBERG, Max; DALIBARD, Valentin a OSINDERO, Simon et al. *Population Based Training of Neural Networks*. Online, arxiv. USA: Cornell University, 2017. Dostupné z: <https://arxiv.org/abs/1711.09846>. [cit. 2025-05-10].
- [5] *MuJoCo*. Online. C2021. Dostupné z: <https://mujoco.org/>. [cit. 2025-05-10].
- [6] TEAM, OpenAI Five. *OpenAI Five defeats Dota 2 world champions*. Online. OpenAI. 2019. Dostupné z: <https://openai.com/index/openai-five-defeats-dota-2-world-champions/#timeline>. [cit. 2025-05-10].
- [7] *Proximal Policy Optimization*. Online. OpenAI. 2017. Dostupné z: <https://openai.com/index/openai-baselines-ppo/>. [cit. 2025-05-10].
- [8] SCHULMAN, John; LEVINE, Sergey; MORITZ, Philipp; JORDAN, Michael I. a ABBEEL, Pieter. *Trust Region Policy Optimization*. Online, arxiv. USA: Cornell University, 2015. Dostupné z: <http://arxiv.org/abs/1502.05477>. [cit. 2025-05-10].
- [9] SCHULMAN, John; WOLSKI, Filip; DHARIWAL, Prafulla; RADFORD, Alec a KLIMOV, Oleg. *Proximal Policy Optimization Algorithms*. Online, arxiv. USA: Cornell University, 2017. Dostupné z: <https://arxiv.org/abs/1707.06347>. [cit. 2025-05-10].
- [10] SCHULMAN, John; MORITZ, Philipp; LEVINE, Sergey; JORDAN, Michael a ABBEEL, Pieter. *High-Dimensional Continuous Control Using Generalized*

- Advantage Estimation*. Online, arxiv. USA: Cornell University, 2018. Dostupné z: <https://arxiv.org/abs/1506.02438>. [cit. 2025-05-10].
- [11] VINYALS, Oriol; BABUSCHKIN, Igor; CHUNG, Junyoung et al. *AlphaStar: Mastering the real-time strategy game StarCraft II*. Online. Google DeepMind. B. r. Dostupné z: <https://deepmind.google/discover/blog/alphastar-mastering-the-real-time-strategy-game-starcraft-ii/>. [cit. 2025-05-10].
- [12] WANG, Guanzhi; XIE, Yuqi a JIANG, Yunfan et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. Online. MineDojo. B. r. Dostupné z: <https://voyager.minedojo.org/>. [cit. 2025-05-10].
- [13] WATKINS, Christopher J. C. H. a DAYAN, Peter. Q-learning. Online. *Machine Learning*. 1992, roč. 8, č. 11, s. 279—292. Dostupné z: <https://doi.org/10.1007/BF00992698>. [cit. 2025-05-10].
- [14] CONTRIBUTORS, Wikipedia. *Deep Blue (chess computer)*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: [https://en.wikipedia.org/w/index.php?title=Deep_Blue_\(chess_computer\)&oldid=1288103584](https://en.wikipedia.org/w/index.php?title=Deep_Blue_(chess_computer)&oldid=1288103584). [cit. 2025-05-10].
- [15] CONTRIBUTORS, Wikipedia. *StarCraft II*. Online. In: Wikipedia: the free encyclopedia. San Francisco (CA): Wikimedia Foundation, 2001-. Dostupné z: https://en.wikipedia.org/w/index.php?title=StarCraft_II&oldid=1286300229. [cit. 2025-05-10].
- [16] ZHANG, Ruize; XU, Zelai a MA, Chengdong et al. *A Survey on Self-play Methods in Reinforcement Learning*. Online, arxiv. USA: Cornell University, 2024. Dostupné z: <https://arxiv.org/abs/2408.01072>. [cit. 2025-05-10].

Příloha A

Název první přílohy

Zde napište text první přílohy nebo jej vložte, např.: `\input{priloha_A.tex}`.